

Manual técnico de arquitectura
GA8-220501096-AA1-EV02

Nombre

Hector Wanderley Palencia Puentes

Instructor

Oscar Fernando Carvajal Castaño

Centro de Teleinformática y Producción Industrial
Análisis y Desarrollo de Software

Ficha: 3186640

2026

SIAPE es un sistema de gestión empresarial para pymes desarrollado con arquitectura cliente-servidor de tres capas. La capa de presentación es una aplicación web React, la capa de lógica de negocio es una API REST en Node.js y la capa de datos es una base de datos MySQL.

1. Tecnologías utilizadas

Frontend:

Tecnología	Versión	Propósito
React	18	Biblioteca para construcción de interfaces
Vite	5	Herramienta de construcción y servidor de desarrollo
React Router	6	Enrutamiento del lado del cliente
Axios	1	Cliente HTTP para consumo de la API
Recharts	2	Visualización de datos en reportes

Backend:

Tecnología	Versión	Propósito
Node.js	18	Entorno de ejecución JavaScript
Express	5	Framework web para la API REST
MySQL2	3	Driver para conexión a MySQL
bcryptjs	3	Hash de contraseñas
jsonwebtoken	9	Generación y verificación de tokens JWT
Resend	4	Envío de correos electrónicos
Jest	29	Framework de testing
Supertest	7	Testing de endpoints HTTP

Base de datos:

Tecnología	Versión	Propósito
MySQL	8.0	Sistema de gestión de base de datos relacional

Despliegue:

Plataforma	Propósito
Vercel	Hosting del frontend
Railway	Hosting del backend y la base de datos
GitHub	Control de versiones y repositorio

2. Arquitectura del sistema

El sistema sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador) en el backend:

Rutas (Routes): Definen los endpoints disponibles y aplican middlewares de autenticación antes de pasar la solicitud al controlador correspondiente.

Controladores (Controllers): Reciben la solicitud HTTP, validan los campos obligatorios y delegan la lógica de negocio al servicio. Retornan la respuesta HTTP al cliente.

Servicios (Services): Contienen toda la lógica de negocio y el acceso a la base de datos mediante consultas SQL directas usando el pool de conexiones MySQL2.

Middlewares: Componentes transversales que se ejecutan antes de los controladores. El principal es verifyToken que valida el JWT en todas las rutas protegidas.

3. Estructura de carpetas

SIAPE/

```
|— frontend/
|  |— src/
|  |  |— components/  # Componentes reutilizables (Header, Layout, etc.)
|  |  |— pages/        # Páginas por módulo (Login, Pedidos, Facturas, etc.)
|  |  |— services/     # Configuración de Axios (api.js)
|  |— index.html
|  |— package.json
|  |— vite.config.js
|— backend/
|  |— src/
|  |  |— config/       # Configuración de la base de datos
|  |  |— controllers/  # Controladores por módulo
|  |  |— middlewares/  # Autenticación y manejo de errores
|  |  |— routes/       # Rutas por módulo e index.js
|  |  |— services/     # Lógica de negocio por módulo
|  |— __tests__/       # Tests de integración por módulo
|  |— scripts/         # Scripts utilitarios
|  |— server.js        # Punto de entrada del servidor
|  |— package.json
|— docs/               # Documentación del proyecto
|— servlets/          # Módulos Java
|— README.md
```

4. Flujo de autenticación

El sistema implementa autenticación en dos pasos:

1. El usuario envía user_name y password al endpoint POST /api/auth/login
2. El backend valida las credenciales comparando el password con el hash bcrypt almacenado
3. Si son correctas genera un código numérico de 6 dígitos y lo guarda en memoria con expiración de 10 minutos
4. El código se envía al correo registrado del usuario usando Resend
5. El usuario envía el código al endpoint POST /api/auth/verify-code
6. El backend valida el código y genera un token JWT firmado con expiración de 8 horas
7. El frontend almacena el token en localStorage y lo envía en el header Authorization: Bearer <token> en todas las solicitudes posteriores
8. El frontend detecta automáticamente la expiración del token y redirige al login

5. Modelo de base de datos

El sistema cuenta con las siguientes tablas principales:

Tabla	Descripción
usuario	Usuarios base del sistema
usuario_trab	Trabajadores con credenciales de acceso
usuario_cli	Clientes de la empresa
proveedor	Proveedores de la empresa
producto	Catálogo de productos
inventario	Stock de productos por bodega
bodega	Espacios de almacenamiento
pedido_proveedor	Pedidos realizados a proveedores
pedido_proveedor_item	Items de cada pedido a proveedor
producto_has_proveedor	Relación entre productos y proveedores
factura	Facturas de venta a clientes
factura_item	Items de cada factura
pago	Pagos registrados por factura
negocio	Información del negocio (singleton)

El diagrama EER completo está disponible en la carpeta docs/ del repositorio.

6. Seguridad

Autenticación: JWT con expiración de 8 horas y verificación en dos pasos por correo.

Contraseñas: Hasheadas con bcrypt con salt aleatorio regenerado en cada cambio.

Autorización: Middleware verifyToken protege todos los endpoints excepto /api/health, /api/auth/login y /api/auth/verify-code.

Roles: El cargo admin en el token JWT determina el acceso a funciones administrativas tanto en frontend como en backend.

Variables sensibles: Las credenciales de BD, JWT y correo están en variables de entorno nunca expuestas en el código.

Integridad: Las operaciones críticas usan transacciones SQL con rollback automático ante fallos.

7. Testing

El sistema cuenta con 81 tests de integración distribuidos en 13 suites usando Jest y Supertest:

Suite	Tests
auth.test.js	4
trabajadores.test.js	6
proveedores.test.js	7
productos.test.js	8
inventario.test.js	7
pedidos-proveedor.test.js	8
clientes.test.js	7
facturas.test.js	7
pagos.test.js	6
bodegas.test.js	8
reportes.test.js	8
negocio.test.js	3
pqrs.test.js	2

Para ejecutar todos los tests: `cd backend && npm test`

8. URLs del sistema

Entorno	Frontend	Backend
Desarrollo	http://localhost:3000	http://localhost:4000
Producción	https://siape-lac.vercel.app	https://siape-production.up.railway.app